UNIVERSIDADE FRANCISCANA
Curso: Ciência da Computação – 2026-01.

Disciplina: Sistemas Operacionais
Professor: Ricardo Frohlich.

Nome (s): José Otávio R. Baggio
Data: 14/04/2026.

Revisão: Escalonamento

1. Quais os principais objetivos do escalonamento de processos?

   R:

   O escalonameto decide quem usa a CPU e quando. Os objetivos dele são:

   • Aumentar a produção do sistema (througput)

   • Diminuir o tempo de resposta aos usuários

   • Manter o processador ocupado o máximo possível

   • Diminuir o tempo médio de espera na fila do processador (turnaroundtime)

2. Diferencie escalonamento preemptivo e escalonamento não-preemptivo.
   R:

   Escolanamento preemptivo: O sistema pode **interromper um processo** antes dele terminar. Há custo associado à coordenação do acesso aos dados compartilhados.
   Ex: Round Robin: Melhor resposta para sistemas interativos, porém há mais overhead (troca de contexto)

   Escalonamento não-preemptivo: O processo **só sai da CPU quando termina ou bloqueia.**

3. Na política de escalonamento FCFS, considere a situação a seguir:
   - Um processo CPU *bound* (P1) obtém e detém a CPU. Muitos processos I/O *bound* terminam sua operação de I/O e passam para o estado de prontos. O P1 passa para a operação de I/O.

   Descreva como acontecerá a utilização da CPU e dos dispositivos. Apresente uma soluções para melhorar o uso dos recursos do sistema de computação.

   R:
   Enquanto P1 usa CPU, os dispositivos de I/O ficam osiosos, quando P1 vai par I/O a CPU pode ficar ociosa esperando, isso é uma má utilização dos recursos da CPU e I/O
   Soluções:
   Usar escalonamento **preemptivo** (ex: Round Robin)

Priorizar processos **I/O-bound**
Usar Shortest Job First (SJF) ou variações
Balancear CPU e I/O para evitar ociosidade

4. Quais são os critérios utilizados para comparar as diferentes políticas de escalonamento?

R:

O escalonamento procura **maximizar** os seguintes critérios

- utilização de CPU: deve ficar o maior tempo possível ocupada
- Throughput: número de processos terminados por unidade de tempo
  - exemplo: 1 processo/hora | 10 processos/segundo

Escalonamento procura **minimizar** os seguintes critérios:

- tempo de retorno: tempo entre a submissão e a conclusão do processo, envolve o tempo na fila de prontos, em execução, usando dispositivos
- tempo de espera: tempo esperando na fila de processos prontos (não afeta operações de I/O)
- tempo de resposta: é o tempo entre a submissão e a primeira resposta do processo (não considera a velocidade do dispositivo de saída)

5. Como é calculado o tempo de retorno e o tempo de espera dos processos?

R:

Tempo de retorno = instante de término − instante de chegada

Tempo de espera = tempo de retorno − ciclos de CPU (tempo de serviço)

6. Considere o seguinte cenário para a Fila de Processos Prontos:

Processo	Instante de chegada	Ciclos de CPU	Tempo de Espera	Tempo de Retorno
P1	0	10		
P2	2	8		
P3	3	5		

Construa o diagrama de tempo ilustrando a execução dos processos conforme a política de escalonamento FIFO, SJF não-preemtivo e SJF preemptivo. Calcule os tempos de espera e de retorno para cada um dos processos em cada uma das políticas.

FIFO:

Processo	Término	Tempo de Retorno	Tempo de Espera
P1	10	10 – 0 = 0	10 – 10 = 0
P2	18	16 – 8 = 10	16 – 8 = 8
P3	23	23 - 3	20 – 5 = 15

Tempo médio de espera = (0+8+15)/3 = 7,67

SJF não-preemtivo(ao liberar a CPU, escolhe o de menor burst disponível):

Processo	Término	Tempo de Retorno	Tempo de Espera
P1	10	10 – 0 = 10	10 – 10 = 0

P3	23	15 – 3 = 12	10 – 3 = 7
P2	15	23 – 2 = 21	15 – 2 = 13

Tempo médio de espera = (0 + 7 + 13) / 3 = 6,66

SJF preemptivo (Sempre executa o processo com **MENOR tempo RESTANTE**):

Processo	Término	Tempo de Retorno	Tempo de Espera
P1	3		
P3	8	8 – 3 = 5	5 – 5 = 0
P1	15	15 – 0 = 15	15 – 10 = 5
P2	23	23 – 2 = 21	21 – 8 = 13

Tempo médio de espera = (0 + 5 + 13) / 3 = 6,00

7. Considere o seguinte conjunto de processos, com duração de surto de CPU expressa em milissegundos, conforme a tabela abaixo. Os processos são considerados como tendo chegado na ordem P1, P2, ..., P5, todos no instante zero.*

Processo	Ciclos CPU	Prioridade
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

a) Desenhe quatro diagramas de Gantt que ilustrem a execução desses processos usando os escalonamentos FIFO, SJF, Prioridade não-preemptivo (um número de prioridade mais baixo implica prioridade mais alta) e RR (quantum=3).

R:

FIFO:

Processo	Chegada	Ciclos CPU	Início	Término/finalização	Retorno	Espera
P1	0	10	0	10	10 – 0 = 10	10 – 10 = 0
P2	0	1	10	11	11 – 0 = 11	11 – 1 = 10
P3	0	2	11	13	13 – 0 = 13	13 – 2 = 11
P4	0	1	13	14	14 – 0 = 14	14 – 1 = 13
P5	0	5	14	19	19 – 0 = 19	19 – 5 = 14

SJF:

Processo	Chegada	Ciclos CPU	Início	Término/finalização	Retorno	Espera
P2	0	1	0	1	1 – 0 = 1	1 - 1 = 0
P4	0	1	1	2	2 – 0 = 2	2 – 1 = 1
P3	0	2	2	4	4 – 0 = 4	4 – 2 = 2
P5	0	5	4	9	9 – 0 = 9	9 – 5 = 4
P1	0	10	9	19	19 – 0 = 19	19 – 10 = 9

Prioridade não-preemptivo:

Processo	Chegada	Ciclos CPU	Prioridade	Início	Término/finalização	Retorno	Espera
P2	0	1	1	0	1	1 – 0 = 1	1 - 1 = 0
P5	0	5	2	1	6	6 – 0 = 6	6 - 5 = 1
P1	0	10	3	6	16	16 – 0 = 16	16 – 10 = 6
P3	0	2	3	16	18	18 – 0 = 18	18 – 2 = 16

P4	0	1	4	18	19	19 – 0 = 19	19 – 1 = 18

Round Robin (Quantum = 3):

Processo	Ciclos CPU	Início	Término/finalização	Retorno	Espera
P1	10	0	19	19 – 0 = 0	19 – 10 = 9
P2	1	1	4	4 – 0 = 4	4 – 1 = 3
P3	2	2	6	6 – 0 = 0	6 – 2 = 4
P4	1	4	7	7 – 0 = 7	7 – 1 = 6
P5	5	9	15	15 – 0 = 15	15 – 5 = 10

b) Qual é o tempo de retorno de cada processo para cada um dos algoritmos de escalonamento do item a)?

c) Qual é o tempo de espera de cada processo para cada um dos algoritmos de escalonamento do item a)?

d) Qual é o tempo médio de espera de cada um dos algoritmos?

FIFO = 9,6 SJF = 4 Prioridade: 8,2 Round Robin: 6,4

8. Em relação ao favorecimento de processos curtos, compare os seguintes algoritmos de escalonamento:

a) FIFO

R:

O FIFO é o pior algoritmo em favor de processos curtos. Pois a ordem de execução é puramente a de chegada, então, pois os processos curtos ficam presos atrás dos processos longos.

b) RR

R:

Favorece parcialmente. Os processos curtos nele terminam bem rapidamente de acordo com o quantum, porém os processos longos vão e voltam periodicamente. Ou seja, tudo depende do tamanho do quantum, quanto menor for melhor será para os processos curtos.

c) Múltiplas Filas com Realimentação

R:

É o que mais favorece processos curtos entre os três. O mecanismo funciona assim: todo processo começa na fila de maior prioridade (quantum pequeno). Se ele usar o quantum completo sem terminar, é rebaixado para uma fila de menor prioridade (quantum maior). Se terminar antes, permanece na fila alta ou sai do sistema.

Resultado: processos curtos terminam rapidamente nas filas de alta prioridade e nunca são rebaixados. Processos longos vão sendo empurrados para filas de baixa prioridade, cedendo espaço para os curtos. É o algoritmo que melhor adapta o escalonamento ao comportamento real de cada processo.

9. O escalonamento por Prioridade sempre resultará em um menor tempo médio de espera dos processos? Justifique.

R:

Não. Ele não garante um menor tempo de espera. O que melhor resulta em um tempo médio de espera é o SJF, pois escolhe o processo com o menor burst. O escalonamento por prioridade só coincidiria com esse resultado se os processos

de maior prioridade fossem também os de menor burst, o que raramente acontece na prática.

Além disso, o escalonamento por prioridade cria um problema grave chamado **starvation**: processos de baixa prioridade podem esperar indefinidamente se chegarem continuamente processos de alta prioridade, o que piora muito o tempo médio de espera do sistema como um todo.

10. O que é *starvation*? Quando pode acontecer? Qual a solução para este problema?
R:
Starvation (inanição) é quando um processo fica esperando pela CPU por um tempo muito longo ou indefinidamente, pois sempre chegam processos com maior prioridade ou menor burst que passam na sua frente.

Quando pode acontecer?
Nos algoritmos que tomam decisões baseadas em características do processo, como o escalonamento por **Prioridade** e o **SJF**. Se chegarem continuamente processos de alta prioridade ou de burst curto, o processo de baixa prioridade ou burst longo nunca consegue a CPU.
Exemplo concreto: imagine um processo de prioridade 10 esperando na fila. Se a cada momento que a CPU libera existe algum processo de prioridade 1, 2 ou 3 disponível, o processo de prioridade 10 simplesmente nunca executa.
Qual a solução?
A solução é o **aging (envelhecimento)**. A ideia é aumentar gradualmente a prioridade de um processo conforme o tempo que ele passa esperando na fila. Assim, mesmo um processo de baixíssima prioridade, depois de esperar tempo suficiente, terá sua prioridade elevada até ser escalado. Isso garante que nenhum processo espera para sempre.

11. O desempenho do Round-Robin depende do tamanho do quantum. Quais as implicações no escalonamento se:

- **for utilizado um quantum imediatamente superior ao tempo necessário para a troca de contexto?**

  R: O sistema vai ficar quase paralisado de tanto alterner entre os processos sem avançar nada.

- **for utilizado um quantum cujo tamanho é imensamente superior a média dos surtos de CPU dos processos do sistema?**
  R: Ele se tornará um FIFO, uma fila normal e assim, não faria sentido utilizar o Round Robin, pois ele é feito para dividir para conquistar.